

Nx Angular Monorepo — Interview Cheatsheet

Folder structure • Responsibilities • Module boundaries • Communication patterns • Restricted access

1. Top-Level Folder Hierarchy

```
my-workspace/
├─ apps/                → deployable, thin shells
│  ├─ admin-portal/      (app project)
│  ├─ admin-portal-e2e/  (Playwright/Cypress)
│  └─ customer-app/
├─ libs/                → all real logic lives here
│  ├─ shared/
│  │  ├─ ui/             (dumb components)
│  │  ├─ utils/          (pure fns, pipes)
│  │  ├─ data-access/    (base API clients)
│  │  └─ types/          (interfaces/models)
│  ├─ auth/
│  │  ├─ feature-login/
│  │  ├─ data-access/
│  │  └─ util/
│  └─ orders/
│     ├─ feature-list/ feature-detail/
│     ├─ data-access/
│     └─ ui/
├─ tools/               (custom generators, scripts)
├─ nx.json              (workspace-level config)
├─ tsconfig.base.json   (path mapping / import aliases)
└─ .eslintrc.json       (module boundary rules)
```

2. Who's Responsible For What

Item	Responsibility
<code>apps/</code>	Deployable artifact — has <code>main.ts</code> , gets built/bundled. Should contain almost no logic , just composition + bootstrap + routing shell.
<code>libs/</code>	All real logic — reusable, independently testable, independently buildable units.
<code>nx.json</code>	Task pipeline config (e.g. <code>build</code> depends on <code>^build</code>), caching rules, plugin registration, named inputs.
<code>project.json</code> (per lib/app)	Targets (<code>build</code> , <code>test</code> , <code>lint</code> , <code>serve</code>) + tags used for module-boundary enforcement.
<code>tsconfig.base.json</code>	Path mappings — defines <i>how</i> one lib imports another, e.g. <code>@myorg/orders/data-access</code> .

Interview line: "Apps are thin shells. If logic lives in `apps/`, you can't reuse it elsewhere, can't test it in isolation, and you lose Nx's affected-graph + caching benefits."

3. Why "libs"? Can You Rename It?

`libs` is just a folder-naming **convention** — not a hardcoded keyword. Nx reads workspace layout from config.

```
// nx.json
{
  "workspaceLayout": {
    "appsDir": "applications",
    "libsDir": "packages"
  }
}
```

- Renaming doesn't break anything — Nx generates new projects wherever configured.
- Existing projects don't auto-move — use `nx g move` to relocate.
- Common alternates: `packages` (npm-style), `modules`.

Q: What does a "lib" represent conceptually?

Any reusable, independently importable unit of code — feature code, UI components, utilities, or state — not just a technical requirement.

4. The 4 Standard Library Types

Type	Purpose	Can import from
------	---------	-----------------

feature-*	Smart components, routed views, business logic orchestration	data-access, ui, util
data-access	State management (NgRx/Signals), API services, effects	util, types
ui	Presentational/dumb components — no business logic	util, types
util	Pure helper functions, pipes, directives, validators	types only

Naming pattern: `libs/<domain>/<type>` → import path `@my-workspace/orders/data-access`

```
nx g @nx/angular:library orders-data-access \
  --directory=libs/orders/data-access \
  --tags=scope:orders,type:data-access
```

5. Public API Pattern (Encapsulation)

Every lib exposes exactly **one** entry point — `index.ts` (the barrel file).

```
libs/orders/data-access/  
└─ src/  
    └─ index.ts      // public API — the ONLY import path  
    └─ lib/  
        └─ order.service.ts // internal — never import directly
```

Why it matters: Deep imports (`libs/orders/data-access/src/lib/order.service.ts`) break encapsulation — refactoring internals then breaks every consumer. Only `index.ts` should be importable, enforced via TS path mapping.

6. How Apps/Libs Communicate

Mechanism	When used
Shared <code>data-access</code> libs	Most common — both apps import same NgRx/Signal store logic + API service. Not runtime "communication," just shared code for consistent behavior.
HTTP calls	True separate deployed apps / microservices / BFF pattern.
BroadcastChannel / localStorage events	Same-origin, cross-tab or micro-frontend messaging.
Module Federation	Apps dynamically load remote modules from each other at runtime; can share a common host for auth/state/routing.
postMessage	Cross-origin / iframe-embedded apps.

Trap Q: If two libs need to talk to each other, how?

They shouldn't — directly coupling libs risks circular dependencies. Extract shared logic into a third lib both depend on.

7. Restricting Access Between Projects

Enforced via **tags + ESLint module boundary rules** — not by folder location.

```
// libs/orders/feature-list/project.json  
{ "tags": ["scope:orders", "type:feature"] }  
  
// libs/shared/ui/project.json  
{ "tags": ["scope:shared", "type:ui"] }
```

```
// .eslintrc.json  
{  
  "rules": {  
    "@nx/enforce-module-boundaries": ["error", {  
      "depConstraints": [  
        { "sourceTag": "scope:orders",  
          "onlyDependOnLibsWithTags": ["scope:orders", "scope:shared"] },  
        { "sourceTag": "type:feature",  
          "onlyDependOnLibsWithTags": ["type:data-access", "type:ui", "type:util"] },  
        { "sourceTag": "type:ui",  
          "onlyDependOnLibsWithTags": ["type:util"] },  
        { "sourceTag": "type:util",  
          "onlyDependOnLibsWithTags": ["type:util"] }  
      ]  
    }  
  }  
}
```

Two independent axes:

- **Scope** (domain isolation) — `orders` can't import from `billing` directly
- **Type** (layering) — `ui` can't import `data-access`; nothing imports `feature` (features are leaves)

Key point: Violations fail **lint**, enforced in CI — not just a code-review convention.

8. Two Apps Share a Feature but Need Restricted Content

Pattern	Approach
Content projection	<code><ng-content select="[adminActions]"></ng-content></code> — App A projects admin buttons in, App B doesn't. Shared component stays unaware of caller.

Config/Input-driven	<code>@Input() permissions: {canDelete, canExport}</code> — each app computes its own permission object in its own data-access/auth lib; shared UI just renders conditionally.
Feature flags	Shared <code>FeatureFlagService</code> (in <code>shared/util</code>) backed by per-app/per-environment flag config.
Structural directive	<code>*appHasRole=" 'admin' "</code> — reusable visibility directive in <code>shared/ui</code> / <code>util</code> , avoids scattering <code>*ngIf</code> everywhere.

Model answer: "Keep the shared component dumb and stateless, pass permission/config as an Input from each app's own auth context, and use a structural directive for repeated visibility checks rather than scattering conditionals across templates."

9. Rapid-Fire Interview Questions

Question	Key Answer
Why many small libs instead of one big lib per app?	Affected/build caching granularity — change one small lib, only that lib + its dependents rebuild/retest.
How does <code>nx affected</code> know what changed?	Compares git diff against dependency graph built from imports + tags + implicit deps.
Why avoid deep imports?	Breaks encapsulation, defeats the public-API boundary, refactoring internals then breaks consumers.
Circular dependency between two libs — what happens?	Nx errors at graph-build time; <code>nx graph</code> visually shows it; build fails.
<code>buildable</code> vs <code>publishable</code> lib?	Buildable = can be built independently for incremental/computation caching. Publishable = additionally packaged correctly to publish to npm.
How to share env-specific config across apps without duplication?	Central config lib (<code>shared/util-config</code>) reading from environment files or injected tokens, consumed by all apps.
What is <code>implicitDependencies</code> vs import-based deps?	Import-based = detected automatically from code. Implicit = manually declared in <code>project.json</code> for non-code links (e.g. app depends on a shared asset/config lib with no direct import).

10. Useful Nx Commands

Command	Use case
<code>nx g @nx/angular:app my-app</code>	Generate new Angular application
<code>nx g @nx/angular:library orders-ui --directory=libs/orders/ui --tags=scope:orders,type:ui</code>	Generate a tagged, scoped library
<code>nx graph</code>	Visualize project dependency graph (spot circular deps, boundary violations)
<code>nx affected:build / test / lint</code>	Only run targets for projects impacted by current git changes
<code>nx g move --project=old-name --destination=libs/new/path</code>	Move/rename a project safely, updates all import paths
<code>nx run-many --target=build --all</code>	Build every project in the workspace
<code>nx lint orders-feature-list</code>	Run ESLint incl. module-boundary checks for one project

Prepared for Angular / Nx monorepo interview revision — covers structure, ownership, boundaries, communication, and restricted-content patterns.